

CSS Flexbox Flex-Shrink

Complete Guide: From Basics to Advanced

Contents

1	Introduction	2
1.1	What is flex-shrink?	2
1.2	Why Use flex-shrink?	2
1.3	The Flex Properties	2
1.4	Default Value	3
2	Understanding flex-shrink	4
2.1	How flex-shrink Works	4
2.2	The Shrinking Calculation	4
2.3	Example: Shrink Distribution	4
2.4	Flex Values Accepted	5
3	Core Usage	6
3.1	flex-shrink: 0 (Don't Shrink)	6
3.1.1	Syntax	6
3.1.2	Characteristics	6
3.1.3	Examples	6
3.2	flex-shrink: 1 (Equal Shrinking)	8
3.2.1	Syntax	8
3.2.2	Characteristics	8
3.2.3	Examples	8
3.3	flex-shrink: 2+ (Proportional Shrinking)	10
3.3.1	Syntax	10
3.3.2	Characteristics	10
3.3.3	Examples	10
4	Practical Applications	13
4.1	Application 1: Responsive Layout	13
4.2	Application 2: Form with Protected Fields	14
4.3	Application 3: Breadcrumb Navigation	14
4.4	Application 4: Toolbar	15
5	Advanced Concepts	17
5.1	flex-shrink with flex-grow	17
5.2	flex-shrink with min-width	17
5.3	Responsive flex-shrink	17
5.4	Content Protection with flex-shrink	18

6	Troubleshooting	19
6.1	Issue 1: Items Not Shrinking	19
6.2	Issue 2: Uneven Shrinking	19
6.3	Issue 3: Content Overflow on Shrink	19
6.4	Issue 4: Items Won't Fit	20
7	Best Practices	21
7.1	When to Use Each Value	21
7.2	Common Patterns	21
7.3	Performance Tips	21
8	Summary: Key Takeaways	22
8.1	Core Concepts	22
8.2	Key Rules	22
8.3	Quick Reference	22
8.4	Comparison: Flex Properties	23

Chapter 1

Introduction

1.1 What is flex-shrink?

The `flex-shrink` property defines how much a flex item will shrink relative to other flex items when there isn't enough space in the flex container. It controls the shrinking factor of an item.

Key Point: `flex-shrink` only applies when items exceed the container width. If items fit perfectly or have extra space, `flex-shrink` has no effect.

1.2 Why Use flex-shrink?

- Control how items shrink when space limited
- Prevent certain items from shrinking
- Create balanced responsive layouts
- Protect important content
- Distribute shrinking proportionally
- Handle dynamic content gracefully
- Simplify complex CSS

1.3 The Flex Properties

`flex-shrink` is part of the flex system:

Property	Purpose
<code>flex-shrink</code>	Shrinking when space limited
<code>flex-grow</code>	Growth when space available
<code>flex-basis</code>	Base size before shrinking
<code>flex</code>	Shorthand for all three

1.4 Default Value

The default `flex-shrink` value is 1, meaning items shrink equally by default.

Chapter 2

Understanding flex-shrink

2.1 How flex-shrink Works

```
/* Default: items shrink equally */  
.flex-item {  
    flex-shrink: 1; /* Default */  
}  
  
/* Item doesn't shrink */  
.flex-item-fixed {  
    flex-shrink: 0;  
}  
  
/* Item shrinks twice as much */  
.flex-item-double {  
    flex-shrink: 2;  
}
```

2.2 The Shrinking Calculation

1. Calculate amount to shrink
2. Sum all flex-shrink factors
3. Divide by shrinking amount
4. Allocate shrinking proportionally

2.3 Example: Shrink Distribution

```
/* Container width: 500px */  
.container {  
    display: flex;  
    width: 500px;  
}  
  
/* Item 1: 200px content, flex-shrink: 1 */
```

```
.item-1 { width: 200px; flex-shrink: 1; }

/* Item 2: 200px content, flex-shrink: 2 */
.item-2 { width: 200px; flex-shrink: 2; }

/* Item 3: 200px content, flex-shrink: 1 */
.item-3 { width: 200px; flex-shrink: 1; }

/* Total: 600px content, need to shrink 100px
   Total shrink factors: 1 + 2 + 1 = 4
   Shrink per factor: 100 / 4 = 25px

   Item 1: 200 - (1 * 25) = 175px
   Item 2: 200 - (2 * 25) = 150px
   Item 3: 200 - (1 * 25) = 175px
*/
```

2.4 Flex Values Accepted

- 0 - Don't shrink
- 1 - Shrink equally (default)
- 2, 3, 4... - Shrink by factor
- Any non-negative number
- Decimal values (1.5, 0.5, etc.)

Chapter 3

Core Usage

3.1 flex-shrink: 0 (Don't Shrink)

Items maintain their size and don't shrink.

3.1.1 Syntax

```
.flex-item {  
    flex-shrink: 0;  
}
```

3.1.2 Characteristics

- Item never shrinks
- Maintains original size
- Other items shrink instead
- May cause overflow
- Good for fixed content

3.1.3 Examples

Example 1: Fixed Navigation

```
<nav class="navbar">  
  <div class="logo">Logo</div>  
  <div class="nav-links">  
    <a href="#">Home</a>  
    <a href="#">About</a>  
    <a href="#">Services</a>  
  </div>  
  <button class="signin">Sign In</button>  
</nav>  
  
.navbar {  
    display: flex;
```



```
    gap: 20px;
    background: #2c3e50;
    padding: 15px;
}

.logo {
    flex-shrink: 0; /* Never shrink */
    width: 100px;
}

.nav-links {
    display: flex;
    gap: 15px;
    flex: 1;
}

.signin {
    flex-shrink: 0; /* Never shrink */
    width: 100px;
}

/* Logo and button don't shrink, links do */
```

Example 2: Fixed Sidebar

```
<div class="layout">
  <aside class="sidebar">
    <nav>Navigation</nav>
  </aside>
  <main class="content">
    <h1>Content</h1>
  </main>
</div>

.layout {
    display: flex;
    gap: 20px;
}

.sidebar {
    width: 250px;
    flex-shrink: 0; /* Never shrink */
    background: #ecf0f1;
}

.content {
    flex: 1;
    background: white;
}

/* Sidebar stays 250px, content shrinks */
```

Example 3: Protected Buttons

```
<div class="toolbar">
```

```
<input type="text" placeholder="Search...">
<button>Save</button>
<button>Export</button>
</div>

.toolbar {
  display: flex;
  gap: 10px;
}

.toolbar input {
  flex: 1;
  padding: 8px;
}

.toolbar button {
  flex-shrink: 0; /* Buttons don't shrink */
  padding: 8px 16px;
  white-space: nowrap;
}

/* Input shrinks, buttons stay fixed */
```

3.2 flex-shrink: 1 (Equal Shrinking)

All items shrink equally when space is limited.

3.2.1 Syntax

```
.flex-item {
  flex-shrink: 1; /* Default */
}
```

3.2.2 Characteristics

- Default behavior
- All items shrink equally
- Fair distribution of shrinking
- Common for flexible layouts
- Balanced responsive design

3.2.3 Examples

Example 1: Equal Shrinking Columns

```
<div class="columns">
  <div class="column">
    <h3>Column 1</h3>
    <p>Content here</p>
  </div>
  <div class="column">
    <h3>Column 2</h3>
    <p>Content here</p>
  </div>
  <div class="column">
    <h3>Column 3</h3>
    <p>Content here</p>
  </div>
</div>

.columns {
  display: flex;
  gap: 20px;
}

.column {
  flex: 1; /* flex-shrink: 1 by default */
  background: white;
  padding: 20px;
  border-radius: 8px;
}

/* All columns shrink equally */
```

Example 2: Card Grid

```
<div class="cards">
  <div class="card">Card 1</div>
  <div class="card">Card 2</div>
  <div class="card">Card 3</div>
</div>

.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}

.card {
  flex: 1 1 200px; /* flex-shrink: 1 */
  background: white;
  padding: 20px;
  border-radius: 8px;
}

/* Cards shrink proportionally */
```

3.3 flex-shrink: 2+ (Proportional Shrinking)

Different items shrink at different rates.

3.3.1 Syntax

```
.flex-item {  
    flex-shrink: 2; /* Shrinks twice as much */  
}
```

3.3.2 Characteristics

- Items shrink proportionally
- Higher values shrink more
- Protected items shrink less
- Useful for priorities
- Protects important content

3.3.3 Examples

Example 1: Protected Main Content

```
<div class="layout">  
    <aside class="sidebar">Sidebar</aside>  
    <main class="content">Main Content</main>  
    <article class="featured">Featured</article>  
</div>  
  
.layout {  
    display: flex;  
    gap: 20px;  
}  
  
.sidebar {  
    width: 200px;  
    flex-shrink: 2; /* Shrinks more */  
}  
  
.content {  
    width: 400px;  
    flex-shrink: 0.5; /* Shrinks less */  
}  
  
.featured {  
    width: 250px;  
    flex-shrink: 1;  
}  
  
/* Main content protected, sidebar shrinks more */
```

Example 2: Priority-Based Shrinking

```
<div class="toolbar">
  <input type="text" placeholder="Search">
  <span class="counter">100 items</span>
  <button class="action">Action</button>
  <button class="menu">Menu</button>
</div>

.toolbar {
  display: flex;
  gap: 10px;
  align-items: center;
}

.toolbar input {
  flex: 1;
  flex-shrink: 1; /* Normal shrinking */
}

.counter {
  flex-shrink: 2; /* Shrinks more, can hide */
}

.action {
  flex-shrink: 0; /* Never shrinks */
}

.menu {
  flex-shrink: 0; /* Never shrinks */
}

/* Input shrinks, counter shrinks more, buttons protected */
```

Example 3: Responsive Navigation

```
<nav class="navbar">
  <div class="logo">Logo</div>
  <div class="nav-links">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Services</a>
    <a href="#">Blog</a>
    <a href="#">Contact</a>
  </div>
</nav>

.navbar {
  display: flex;
  gap: 20px;
  align-items: center;
}

.logo {
  flex-shrink: 0; /* Never shrinks */
}
```

```
        white-space: nowrap;
    }

    .nav-links {
        display: flex;
        gap: 15px;
        flex: 1;
        flex-shrink: 1;
    }

    .nav-links a {
        flex-shrink: 1;
        white-space: nowrap;
    }

    /* Navigation can shrink, logo can't */
```

Chapter 4

Practical Applications

4.1 Application 1: Responsive Layout

```
<div class="layout">
  <header>Header</header>
  <div class="body">
    <aside class="sidebar">Sidebar</aside>
    <main class="content">Content</main>
  </div>
  <footer>Footer</footer>
</div>

.layout {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}

header {
  flex-shrink: 0; /* Fixed size */
  height: 60px;
}

.body {
  flex: 1;
  display: flex;
  gap: 20px;
}

.sidebar {
  width: 250px;
  flex-shrink: 0; /* Fixed width */
}

.content {
  flex: 1;
  flex-shrink: 1; /* Shrinks when needed */
}
```

```
footer {  
    flex-shrink: 0; /* Fixed size */  
    height: 60px;  
}  
  
/* Header/footer fixed, content area flexible */
```

4.2 Application 2: Form with Protected Fields

```
<form class="form">  
    <div class="form-row">  
        <input type="text" placeholder="Search...">  
        <button class="search">Search</button>  
        <button class="advanced">Advanced</button>  
    </div>  
</form>  
  
.form {  
    display: flex;  
    gap: 10px;  
}  
  
.form input {  
    flex: 1;  
    flex-shrink: 1;  
    padding: 8px;  
}  
  
.form .search {  
    flex-shrink: 0; /* Button never shrinks */  
    padding: 8px 16px;  
    white-space: nowrap;  
}  
  
.form .advanced {  
    flex-shrink: 0; /* Button never shrinks */  
    padding: 8px 16px;  
    white-space: nowrap;  
}  
  
/* Input shrinks, buttons stay fixed */
```

4.3 Application 3: Breadcrumb Navigation

```
<nav class="breadcrumb">  
    <a href="/">Home</a>  
    <span class="separator">></span>  
    <a href="/products">Products</a>  
    <span class="separator">></span>  
    <a href="/products/electronics">Electronics</a>
```



```
<span class="separator"></span>
<span class="current">Laptop</span>
</nav>

.breadcrumb {
  display: flex;
  gap: 5px;
  align-items: center;
  padding: 15px;
  background: #ecf0f1;
}

.breadcrumb a {
  flex-shrink: 0; /* Links don't shrink */
  white-space: nowrap;
  color: #3498db;
}

.breadcrumb .separator {
  flex-shrink: 0; /* Separators don't shrink */
}

.breadcrumb .current {
  flex: 1;
  flex-shrink: 1; /* Current item can shrink */
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

/* Links and separators protected, current item shrinks */
```

4.4 Application 4: Toolbar

```
<div class="toolbar">
  <div class="toolbar-group">
    <button>Bold</button>
    <button>Italic</button>
    <button>Underline</button>
  </div>

  <input type="text" placeholder="Search...">

  <div class="toolbar-group">
    <button>Save</button>
    <button>Share</button>
  </div>
</div>

.toolbar {
  display: flex;
  gap: 10px;
```

```
    align-items: center;
    padding: 10px;
    background: #ecf0f1;
}

.toolbar-group {
    display: flex;
    gap: 5px;
    flex-shrink: 0; /* Groups don't shrink */
}

.toolbar-group button {
    padding: 8px 12px;
    flex-shrink: 0; /* Buttons don't shrink */
    white-space: nowrap;
}

.toolbar input {
    flex: 1;
    flex-shrink: 1; /* Input can shrink */
    padding: 8px;
}

/* Buttons protected, input flexible */
```

Chapter 5

Advanced Concepts

5.1 flex-shrink with flex-grow

```
.item {
    flex-grow: 1;      /* Grow when space available */
    flex-shrink: 1;    /* Shrink when space limited */
    flex-basis: 200px;
}

/* Shorthand */
.item {
    flex: 1 1 200px; /* grow shrink basis */
}

/* Flexible in both directions */
```

5.2 flex-shrink with min-width

```
.item {
    flex-shrink: 1;
    min-width: 100px; /* Don't shrink below 100px */
}

/* Item shrinks but respects minimum */
```

5.3 Responsive flex-shrink

```
.sidebar {
    width: 250px;
    flex-shrink: 1;
}

/* Mobile: Allow sidebar to shrink */
@media (max-width: 768px) {
    .sidebar {
```

```
        flex-shrink: 2; /* Shrinks more */
    }
}

/* Desktop: Protect sidebar */
@media (min-width: 1200px) {
    .sidebar {
        flex-shrink: 0; /* Doesn't shrink */
    }
}
```

5.4 Content Protection with flex-shrink

```
.container {
    display: flex;
    gap: 10px;
}

.content {
    flex: 1;
    flex-shrink: 1;
}

.content p {
    min-width: 0; /* Allow text wrapping */
    overflow: hidden;
    text-overflow: ellipsis;
}

/* Content can shrink without overflow */
```

Chapter 6

Troubleshooting

6.1 Issue 1: Items Not Shrinking

Problem: Items don't shrink even with `flex-shrink: 1`.

Solution: Check if `flex-shrink: 0` is set elsewhere.

```
/* WRONG: flex-shrink: 0 prevents shrinking */
.item {
  width: 300px;
  flex-shrink: 0; /* Won't shrink */
}

/* CORRECT: Use flex-shrink: 1 */
.item {
  width: 300px;
  flex-shrink: 1; /* Will shrink */
}
```

6.2 Issue 2: Uneven Shrinking

Problem: Items with same `flex-shrink` shrink differently.

Solution: Check content size and basis.

```
/* Items with different content */
.item-1 { width: 200px; flex-shrink: 1; }
.item-2 { width: 400px; flex-shrink: 1; }

/* Item 2 appears to shrink more due to size */

/* Solution: Adjust flex-shrink proportionally */
.item-1 { width: 200px; flex-shrink: 2; }
.item-2 { width: 400px; flex-shrink: 1; }
```

6.3 Issue 3: Content Overflow on Shrink

Problem: Content overflows when items shrink.

Solution: Add `min-width: 0` to allow content wrapping.

```
.item {
  flex-shrink: 1;
  min-width: 0; /* Allow text wrapping */
}

.item-text {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

/* Content wraps instead of overflowing */
```

6.4 Issue 4: Items Won't Fit

Problem: Even with shrinking, items don't fit.

Solution: Use `flex-wrap: wrap` or overflow handling.

```
.container {
  display: flex;
  flex-wrap: wrap; /* Allow wrapping */
  gap: 10px;
}

.item {
  flex-shrink: 1;
  min-width: 100px; /* Minimum size */
}

/* Items wrap instead of overlapping */
```

Chapter 7

Best Practices

7.1 When to Use Each Value

- **0:** Fixed size items (buttons, logo)
- **1:** Regular flexible items (default)
- **2+:** Expendable content (sidebars, details)

7.2 Common Patterns

```
/* Protected content */
.logo { flex-shrink: 0; }

/* Flexible content */
.search { flex: 1; flex-shrink: 1; }

/* Protected buttons */
.button { flex-shrink: 0; white-space: nowrap; }

/* Sidebar that can shrink */
.sidebar { width: 250px; flex-shrink: 1; }

/* Main content flexible */
.main { flex: 1; flex-shrink: 1; }
```

7.3 Performance Tips

- flex-shrink is performant
- Combine with min-width for stability
- Test at different screen sizes
- Use with responsive design

Chapter 8

Summary: Key Takeaways

8.1 Core Concepts

- `flex-shrink` controls shrinking behavior
- Default value is 1
- Higher values shrink more
- 0 prevents all shrinking
- Proportional to other items

8.2 Key Rules

1. Default `flex-shrink: 1`
2. Only positive values allowed
3. Shrinking is proportional
4. Use 0 for fixed items
5. Combine with `flex-basis`
6. Works with `min-width`
7. Responsive with media queries
8. Very efficient

8.3 Quick Reference

```
/* Don't shrink */  
.fixed { flex-shrink: 0; }  
  
/* Shrink equally */  
.flexible { flex-shrink: 1; }
```



```
/* Shrink more */
.collapsible { flex-shrink: 2; }

/* Complete flexibility */
.item { flex: 1 1 200px; }

/* Protected layout */
.sidebar { flex-shrink: 0; width: 250px; }
.content { flex: 1 1 0; }
```

8.4 Comparison: Flex Properties

Property	Purpose
flex-grow	Growth when space available
flex-shrink	Shrinking when space limited
flex-basis	Base size
flex	Shorthand (grow shrink basis)